Studio Nx

Backend(FastAPI/Python) 49 — purge_withdrawn_accounts()

2026-10-08

Backend(FastAPI/Python) 49 — `purge_withdrawn_accounts()`

항목	내용
파일	`studionx/backend/app/workers/retention.py`
작업	P1-09 비밀번호 변경 · 탈퇴 — 탈퇴 3일 뒤 개인 정보 정리 · PRD 01 FR-108 · 개인정보 처리방침 §4
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	48 `DELETE /users/me` · 50 `InquiryCreateRequest` (P1-12)

1. 하는 일

하루 한 번 돌리는 **정리 작업**. 탈퇴한 지 3일이 지난 계정에서 개인 정보를 비운다. 처리방침(privacy.html §4)에 적은 "탈퇴하면 3일 보관 뒤 비움" 약속을 실제로 지키는 곳.

칸	정리 전	정리 후
`email`	`old.gone@example.com`	`deleted+85@withdrawn.invalid`
`nickname`	`old_gone`	`deleted_user_0000000085`
`password_hash`	`$2b$12$…`	`!` (어떤 비밀번호와도 맞지 않음)
`last_login_at`	마지막 로그인 시각	`None`
`profile_image_path` · 사진 파일	(남아 있으면)	`None` · 파일 삭제
`status` · `deleted_at` · 행 · 프로젝트		**그대로**

계정	결과
탈퇴 4일 · 정확히 3일	정리
탈퇴 2일	그대로 — 3일 동안은 재가입을 막는 데 이메일 · 닉네임을 씀
활동 회원	그대로
이미 다 비운 계정	건너뜀 (세지도 않음)
누가 그 이메일로 가입해 이메일 · 닉네임만 비켜 둔 계정	남은 비밀번호 해시 등을 마저 비움

돌려주는 값 = 이번에 비운 계정 수.

2. 언제 쓰나

실행	언제
`python -m app.workers.retention` 의 `main()`	하루 한 번 (운영체제 예약 작업 — 배포할 때 정함). **`main()` 은 P1-12** — 문의 정리(`purge_old_inquiries`)와 함께 부르므로 그 함수를 만들 때 구현

탈퇴 회원의 시간표

시점	상태	하는 곳
탈퇴 순간	`DELETED`, 사진 삭제, 로그인 · 재발급 차단	46 · 48
~ 3일	같은 이메일 · 닉네임으로 가입 불가 (409 `REJOIN_TOO_SOON`)	P1-04

시점	상태	하는 곳
3일 뒤 누가 그 값을 쓰려 함	이메일 · 닉네임만 바로 비킴	P1-04 `release_withdrawn`
3일 뒤 첫 정리 작업	**나머지까지 모두 비움**	**49**

추적 방지 (2026-10-07 결정 ②)

비켜 둔 값에는 회원 번호만 남고, 원래 이메일 · 닉네임 · 비밀번호 해시 · 마지막 로그인 · 사진이 남지 않는다. ① 다른 사람에게 보이는 곳은 'Deleted user', ③ 번호로 조회하는 공개 API 없음(P1-08 사진 주소 포함)과 함께 쓴다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models import User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`datetime` 줄 · `Session` 줄 말고는 모두 이번에 추가했다.

상수

```
# 다 비운 계정의 비밀번호 칸 값 — bcrypt 해시 모양이 아니라 verify_password 가 늘 False
# (칸이 NOT NULL 이라 빈칸 대신)
WIPED_PASSWORD_HASH = "!"
```

함수

```
def purge_withdrawn_accounts(db: Session, now: datetime) -> int:
    if now.tzinfo is not None:
        now = now.astimezone(UTC).replace(tzinfo=None)
    cutoff = now - timedelta(days=REJOIN_WAIT_DAYS)
    users = db.scalars(
        select(User).where(
            User.status == UserStatus.DELETED,
            User.deleted_at <= cutoff,
            or_(
                ~User.email.startswith("deleted+", autoescape=True),
                ~User.nickname.startswith("deleted_user_", autoescape=True),
                User.password_hash != WIPED_PASSWORD_HASH,
                User.last_login_at.is_not(None),
                User.profile_image_path.is_not(None),
            ),
        )
    ).all()
```

```
    photos = []
    for user in users:
        release_withdrawn(db, user)
        if user.profile_image_path:
            photos.append(user.profile_image_path)
        user.password_hash = WIPED_PASSWORD_HASH
        user.last_login_at = None
        user.profile_image_path = None
    db.commit()

    storage_dir = get_settings().storage_dir
    for relative in photos:
        (storage_dir / relative).unlink(missing_ok=True)
    return len(users)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import or_, select`	`or_(A, B, …)` = SQL 의 `A OR B OR …`
`from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn`	3일 · 비켜 두기 — 가입 쪽(P1-04)과 **같은 값 · 같은 함수**를 씀

식	뜻
`now.astimezone(UTC).replace(tzinfo=None)`	한국 시각처럼 시간대가 붙은 값을 UTC 로 바꾸고 표시를 뗌 — DB 는 UTC
`User.email.startswith("deleted+", autoescape=True)`	SQL `LIKE 'deleted+%'`. `autoescape` 는 값 안의 `_` · `%` 를 글자 그대로 다룸 (`LIKE` 에서 `_` 는 "아무 한 글자")
`~조건`	NOT
`.is_not(None)`	`IS NOT NULL`
`db.scalars(…).all()`	조건에 맞는 `User` 들을 목록으로

4. 흐름

1. 기준 시각 = 지금 − 3일 (UTC).
2. 탈퇴했고 기준보다 오래됐으며 **아직 남은 것이 하나라도 있는** 계정을 찾는다.
3. 각 계정: 이메일 · 닉네임 비키기 → 사진 경로 기억 → 비밀번호 · 마지막 로그인 · 사진 경로 비우기.
4. 한 번에 저장.
5. 저장 뒤 사진 파일 삭제 (이미 없으면 넘어감).
6. 비운 수를 돌려준다.

5. 왜 이렇게

선택	이유
"남은 것이 하나라도 있는" 행만	매일 돌려도 같은 행을 다시 세지 않는다. 가입 때 이메일 · 닉네임만 비켜 둔 행도 빠뜨리지 않는다 (템플릿은 "이메일이 `deleted+` 가 아닌 행" 만 — 그러면 그런 행의 비밀번호 해시가 영영 남는다)
`password_hash = "!"`	칸이 `NOT NULL` 이라 비울 수 없다. `"!"` 는 bcrypt 해시 모양이 아니어서 어떤 비밀번호로도 로그인할 수 없다 (44 시험에서 확인)

선택	이유
`release_withdrawn` 을 그대로 씀	비켜 두는 값이 가입 쪽과 늘 같다. 여러 번 불려도 같은 결과
`deleted_at` · 행을 남김	프로젝트(owner_id)가 이 행을 가리킨다. 탈퇴 시각은 기록으로 필요
사진 파일은 저장 **뒤** 삭제	38 · 39 와 같은 순서 — 저장이 실패하면 파일도 남는다
`now` 를 받음 (안에서 시각을 재지 않음)	시험에서 시각을 정해 넣을 수 있다 ("정확히 3일" 같은 경계)
시간대가 있는 `now` 도 받음	실행하는 곳이 한국 시각을 넘겨도 틀리지 않게

알아 둘 점 — 색인: `status` · `deleted_at` 조건으로 회원 표를 훑는다. 하루 한 번이라 회원이 많아도 부담이 적다.

6. 시험 (`tests/test_retention.py` 새 파일 · 8개, 실제 MySQL + 임시 폴더)

시험	확인한 것
탈퇴 4일 (사진 · 로그인 기록 · 프로젝트 있음)	1 · 이메일 · 닉네임 비킴 · 해시 `!` · 마지막 로그인 · 사진 경로 `None` · 파일 삭제 · 상태 · 탈퇴 시각 · 프로젝트 그대로
탈퇴 2일 · 활동 회원	0 · 그대로
정확히 3일	정리
두 번 돌리기	1 → 0
가입 때 이메일 · 닉네임만 비켜 둔 행	1 · 해시 · 로그인 기록도 비움
사진 파일이 이미 없음	오류 없이 정리
한국 시각 21:00 (= UTC 12:00) 로 실행	정확히 3일로 계산해 정리
정리 뒤	옛 비밀번호로 로그인 401, 옛 이메일 · 닉네임으로 가입 가능

전체 시험 352개 통과, `ruff` 검사 통과.